

Computação Paralela e Distribuída

Aula de Laboratório Nº 3

Coordenação de Engenharia Informática

Departamento de Engenharias e Tecnologias

Instituto Superior Politécnico de Tecnologias e Ciências

Objectivos

- Analisar o desempenho do programa OpenMP
- Optimizar o desempenho de programa OpenMP

1. Introdução

Ao longo desta aula o estudante é encorajado a consultar a documentação em <http://www.openmp.org/wp-content/uploads/openmp-4.5.pdf> sempre que tiver dúvidas sobre as directivas ou APIs do OpenMP e as referências para o processo de instalação e configuração da Intel VTune.

A criação de perfil (*Profiling*) é o primeiro passo para optimizar e ajustar a sua aplicação. Permite entender onde a maior parte do tempo de execução é gasta, ou seja, oferece uma compreensão das características de desempenho da aplicação e identifica partes do código que apresentam oportunidades de melhoria. Assim sendo, permite encontrar pontos críticos (*hotspots*) e gargalos (*bottlenecks*) em sua aplicação para que possa decidir onde concentrar seus esforços de optimização.

Uma ferramenta de Profiling interessante é o Intel Vtune.

2. Intel VTune

O Intel VTune Profiler é uma ferramenta de análise de desempenho geral para plataformas baseadas em x86 com suporte para C/C++, Python, Java e outros modelos de programação, como ambientes paralelos OpenMP e MPI. Os desenvolvedores podem usar o Intel VTune Profiler para identificar funções demoradas, medir sobrecargas em programas multithread, colectar e relatar métricas de hardware (por exemplo, consumo de energia, acessos à memória e instruções por relógio) em CPUs, GPUs e FPGAs, entre outras análises. O profiler pode ser executado usando uma interface gráfica (GUI) ou uma interface de linha de comando (CLI). Para obter mais informações, consulte o Guia do utilizador do Intel VTune Profiler [1].

A análise baseada em software funciona em processadores Intel e AMD, mas o perfil baseado em hardware e outros recursos avançados funcionam apenas com hardware Intel. Como alternativa, o AMDµProf pode ser usado para criar perfis de códigos em execução em máquinas com CPUs AMD, no entanto, é bastante limitado em termos de recursos e usabilidade.

3. Utilização

Antes de mais é preciso descarregar a ferramenta e instalar em seu computador (acesse o link: [Get Intel® VTune™ Profiler](https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler-download.html)).

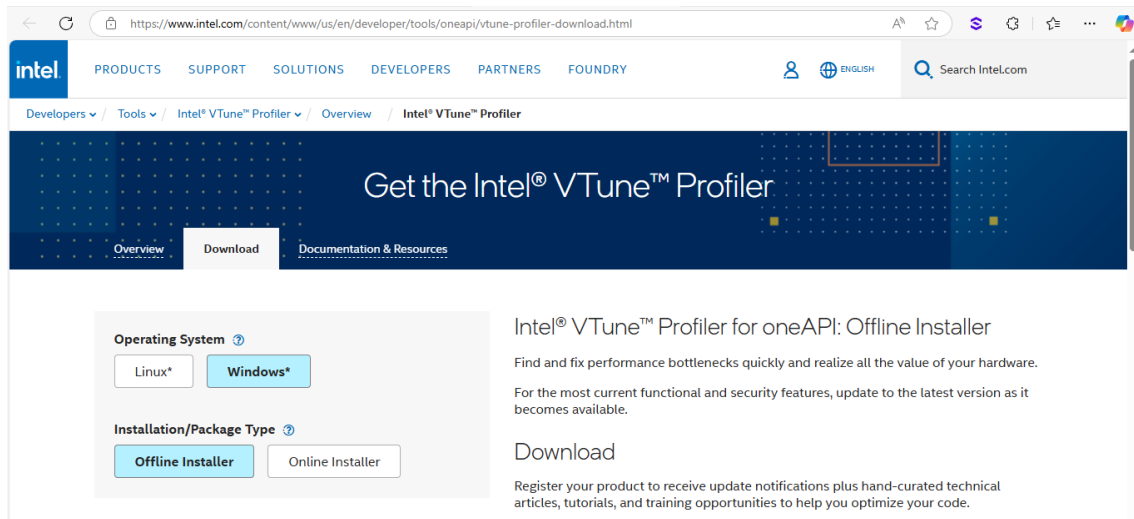


Figure 1: Site oficial da ferramenta para descarregar a aplicação Intel VTune.

O programa Intel VTune está localizado na pasta <caminho de instalação>/bin64 (em máquinas Linux, o caminho de instalação padrão é /opt/intel/oneapi/vtune/latest/) e pode executar:

- **vtune-gui** para iniciar a interface gráfica (secção 3.1, **recomendado**); ou
- **vtune** para usar o modo CLI do profiler (secção 3.2).

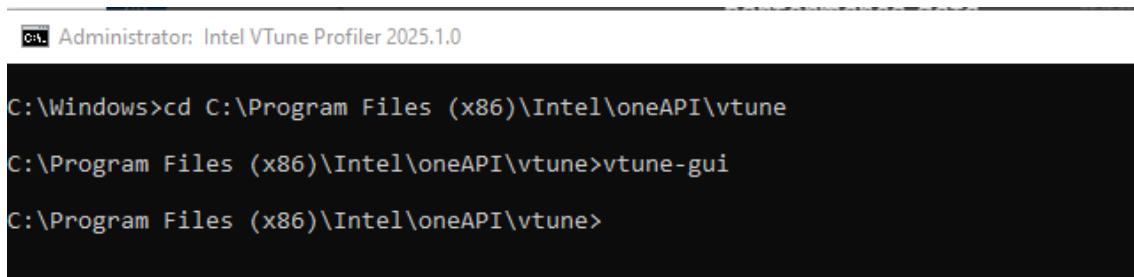


Figure 2: Iniciar a ferramenta a partir da linha de comando.

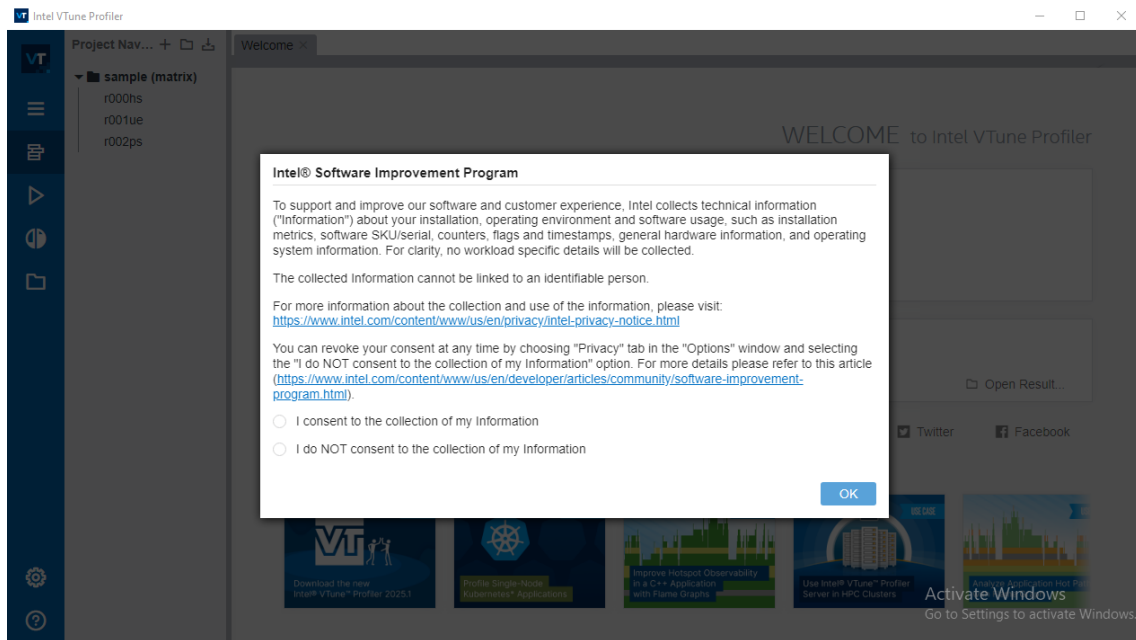


Figure 3: Intel VTune Profiler - Primeira abertura da ferramenta.

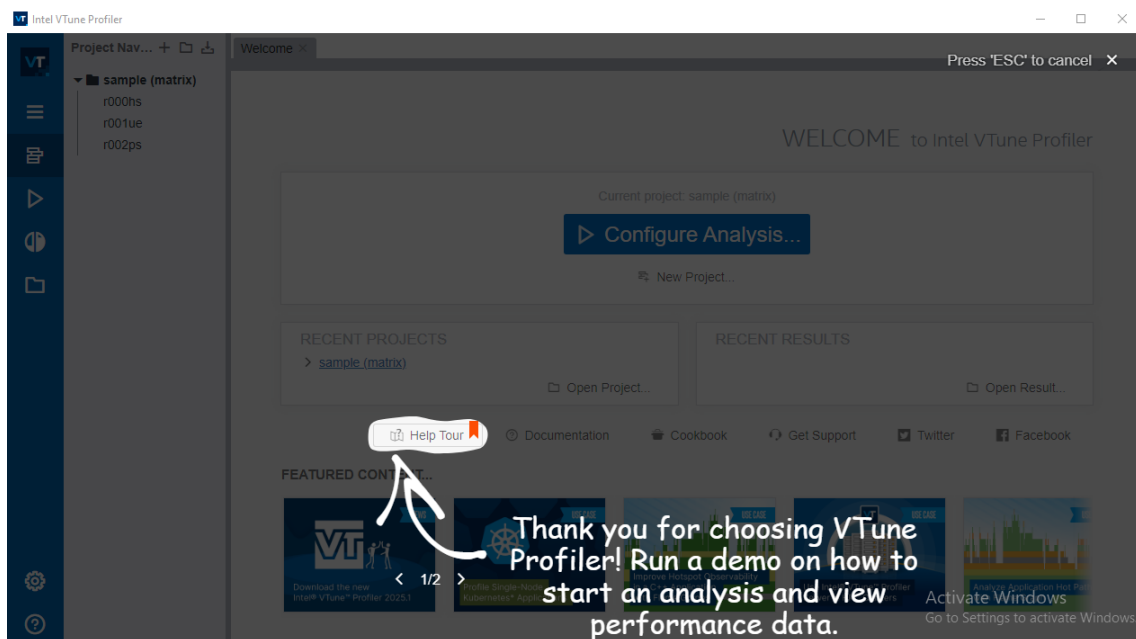


Figure 4: Intel VTune Profiler - Indicação do tutorial para iniciantes.

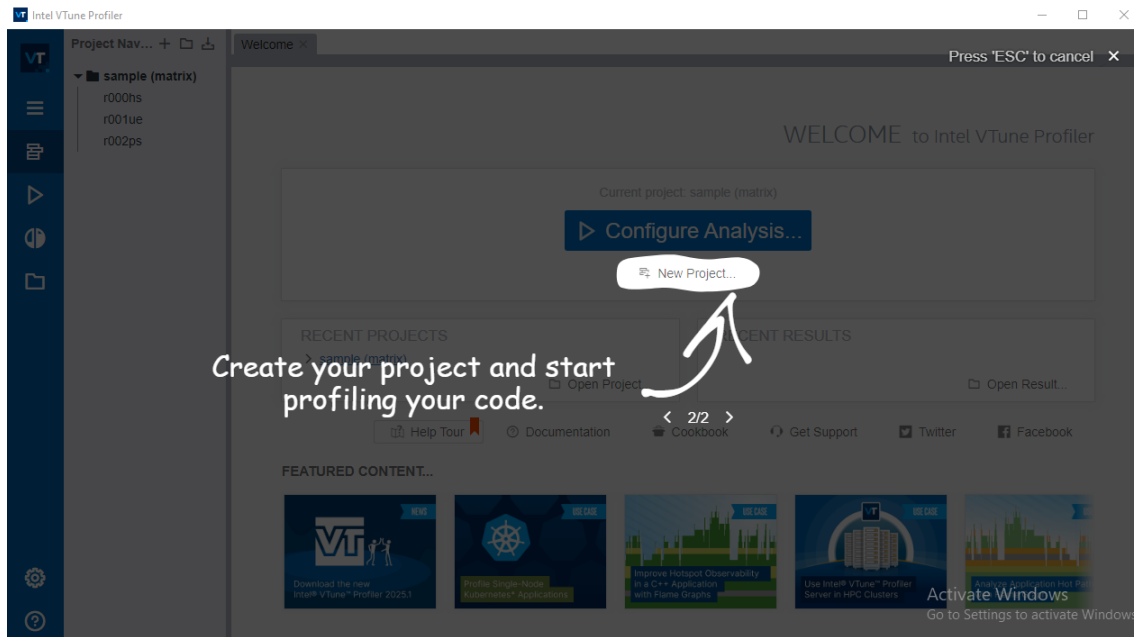


Figure 5: Intel VTune Profiler - Carregar um projecto para analisar o perfil.

Para associar correctamente as medições ao seu código-fonte, lembre-se de usar o flag de debug `-g` ao compilar o programa. O Intel VTune Profiler também precisa dos símbolos de debug do OpenMP, MPI ou de outras bibliotecas. Se tiver problemas para obter os símbolos de debug para libgomp (biblioteca OpenMP do GCC), consulte o Apêndice A.

Também pode adicionar a pasta `<caminho de instalação>/bin64` ao seu PATH para evitar ter que navegar até ela sempre que quiser iniciar o Intel VTune. Por exemplo, adicione a seguinte linha ao final do arquivo `.bashrc` no seu directório home:

```
PATH=$PATH:<instalation path>/bin64/; export PATH
```

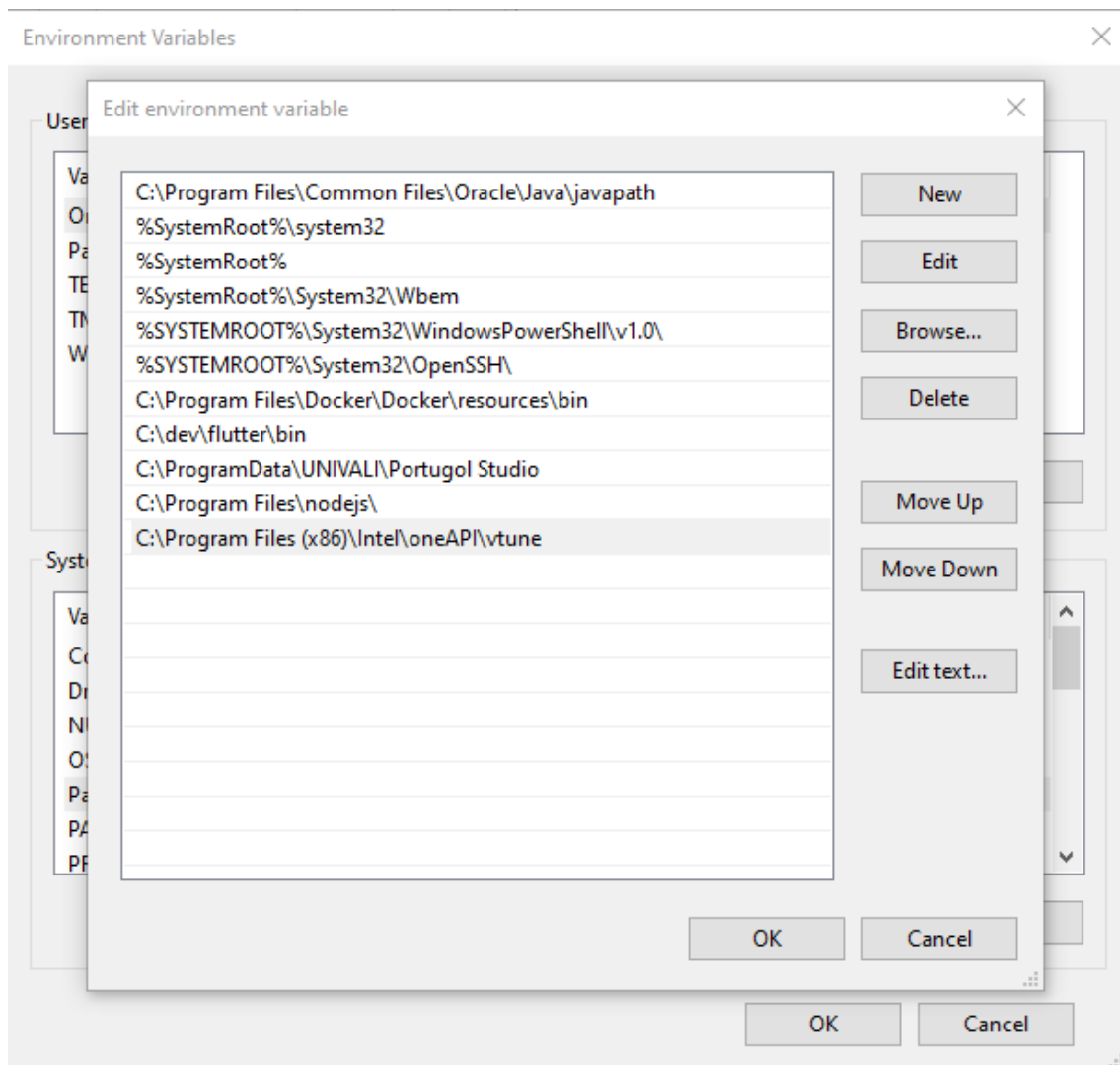


Figure 6: Configuração da variável de ambiente no Windows.

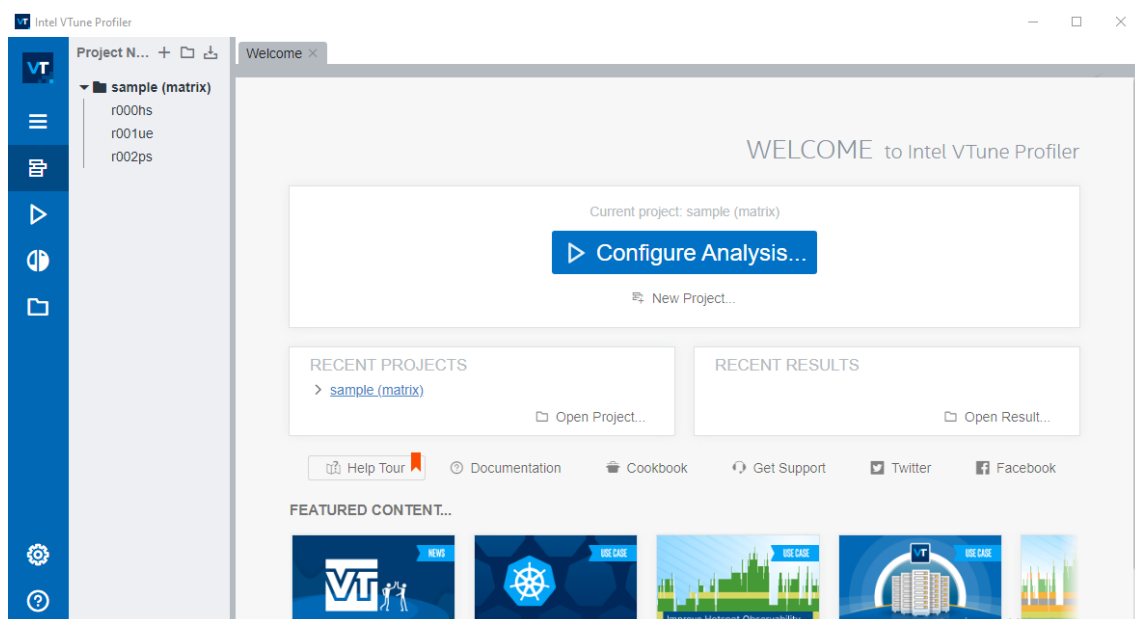


Figure 7: Intel VTune Profiler - Configuração de análise.

Comece por seguir o tutorial de exemplo disponível na ferramenta.

3.1. Interface Gráfica

Após iniciar a Interface Gráfica do Utilizador (GUI) do Intel VTune, crie um Novo Projecto. Se desejar reutilizar um projecto existente, clique em Configurar Análise... Em seguida, especifique o programa de destino e seus argumentos e selecione a análise desejada (2). Em seguida, inicie a criação de perfil (3).

Após a conclusão da criação de perfil, o Intel VTune gerará um resumo dos resultados. Use as guias para alterar a forma como as informações são apresentadas. Por exemplo, na análise de Threading, a visualização Bottom Up mostra o tempo efectivo, o tempo de spin e o tempo de overhead para cada rotina na pilha de chamadas, bem como a linha do tempo de cada thread, enquanto a visualização Top-down Tree mostra informações semelhantes, mas organizadas como uma árvore de chamadas.

O Intel VTune pode apresentar uma explicação de uma determinada métrica ao passar o mouse sobre ela. Clicar na aba "Bem-vindo" retornará à página inicial. Os resultados de cada criação de perfil serão guardados no seu projecto (organizados como pastas no painel esquerdo) e poderão ser reabertos no futuro sem a necessidade de executar a aplicação inteira novamente.

3.2. Interface de Linha de Comandos

Embora usar uma interface gráfica seja mais fácil e intuitivo, às vezes só tem acesso ao terminal em uma máquina. Nesse caso, pode colectar (e reportar) as métricas de desempenho do seu programa usando o modo CLI do profiler. Por exemplo, pode executar a análise de Hotspots para a sua aplicação e guardar os resultados em <dir> da seguinte forma:

```
$ vtune -collect hotspots -result-dir <dir> ./<application>
```

Pode usar o terminal novamente para gerar um relatório (**vtune** -report) ou importar os resultados para a Interface Gráfica do Utilizador (GUI) do Intel VTune na sua máquina local para visualizá-los (copie <dir> para a máquina local e, na aba "Bem-vindo", use "Abrir Resultado"). Consulte o Guia do Utilizador do Intel VTune para obter mais detalhes ou use **vtune** --help.

4. Exercícios

4.1. Hotspots

O objectivo da análise de Hotspots é ajudar os desenvolvedores a entender o fluxo de suas aplicações e identificar em quais secções do código seus programas dedicam mais tempo (ou seja, os hotspots do programa). Usaremos sua implementação serial do projecto para nos familiarizarmos com as funções básicas desta ferramenta.

1. Compile seu código com informações de *debugging* (*flag* -g no caso do gcc).
2. Execute a análise de Hotspots com *Sampling* em Modo de Utilizador.

3. Identifique as secções de código que consomem mais tempo (tabela em Principais Hotspots). Clicar no nome da função abrirá a visualização *Bottom-Up*. Se clicar duas vezes em uma função, mostrará sua localização no *source file*.
4. Alterne para a visualização *Flame Graph*. Cada bloco corresponde a uma rotina na pilha de chamadas e seu tamanho representa o tempo de execução relativo. Se clicar em um bloco, será expandido, permitindo que veja mais detalhes. Clique em Total para retornar à visualização completa. Observe que rotinas com o mesmo nome e profundidade na pilha de chamadas podem ser mescladas.

4.2. Threading

Com a análise de threading, os desenvolvedores podem medir a eficiência com que o programa usa as unidades de processamento disponíveis e identificar ineficiências no multithread causadas por desequilíbrio de carga, sincronização excessiva, etc. Usaremos essa análise para comparar diferentes implementações paralelas de uma rotina de multiplicação de matrizes.

1. Abra `mult-sync.c`, `mult-redux.c` e `mult-good.c`. Quais problemas consegue identificar em `mult-sync.c` e `mult-redux.c`?
2. Execute a análise de threading com Sampling em modo de utilizador para `mult-sync`. Porquê que a utilização efectiva da CPU é tão baixa?
3. Mude para a visualização Top-down Tree e expanda a árvore da seguinte forma: Total > clone > start thread > [OpenMP worker]. Clique duas vezes em `main.omp fn.0` (ou seja, a função principal) para abrir o código-fonte. Quais directivas do OpenMP apresentam alto overhead? Qual é o principal problema nesta implementação?
4. Repita um procedimento semelhante para `mult-redux`. Onde está o ponto de sincronização no código? Porquê que o programa gastou tanto tempo nele?
5. Agora, execute a análise de threading para `mult-good`. Porquê que essa implementação é mais eficiente?

Referências

1. <https://www.intel.com/content/www/us/en/docs/vtune-profiler/cookbook/2025-0/overview.html>
2. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler-download.html>
3. <https://www.intel.com/content/www/us/en/docs/vtune-profiler/installation-guide/2025-0/windows.html>
4. <https://www.intel.com/content/www/us/en/developer/articles/system-requirements/vtune-profiler-system-requirements.html>

APÊNDICE

A. Símbolos de debug

Por padrão, as bibliotecas em muitas distribuições Linux não contêm os símbolos de debug. O Intel VTune precisa desses símbolos para identificar correctamente as directivas e funções do OpenMP. No Ubuntu (e no Windows Subsystem for Linux, WSL), os símbolos de debug para libgomp podem ser adicionados da seguinte forma:

1. Crie `/etc/apt/sources.list.d/ddebs.list` contendo os repositórios com os pacotes de símbolos de debug:

```
printf "deb http://ddebs.ubuntu.com %s main restricted\n" $(lsb_release -cs) {,-updates,-security,-proposed} | sudo tee -a /etc/apt/sources.list.d/ddebs.list
```

2. Importe a chave de assinatura do arquivo de símbolos de debug do servidor Ubuntu:

```
sudo apt install ubuntu-dbgsym-keyring
```

3. Actualize sua lista de pacotes:

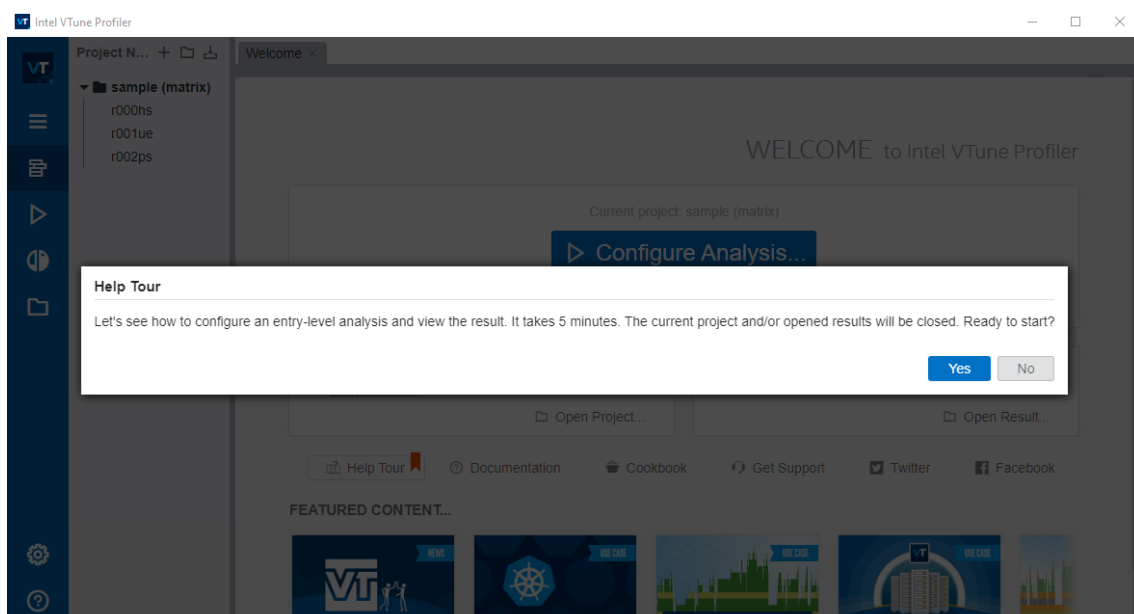
```
sudo apt-get update
```

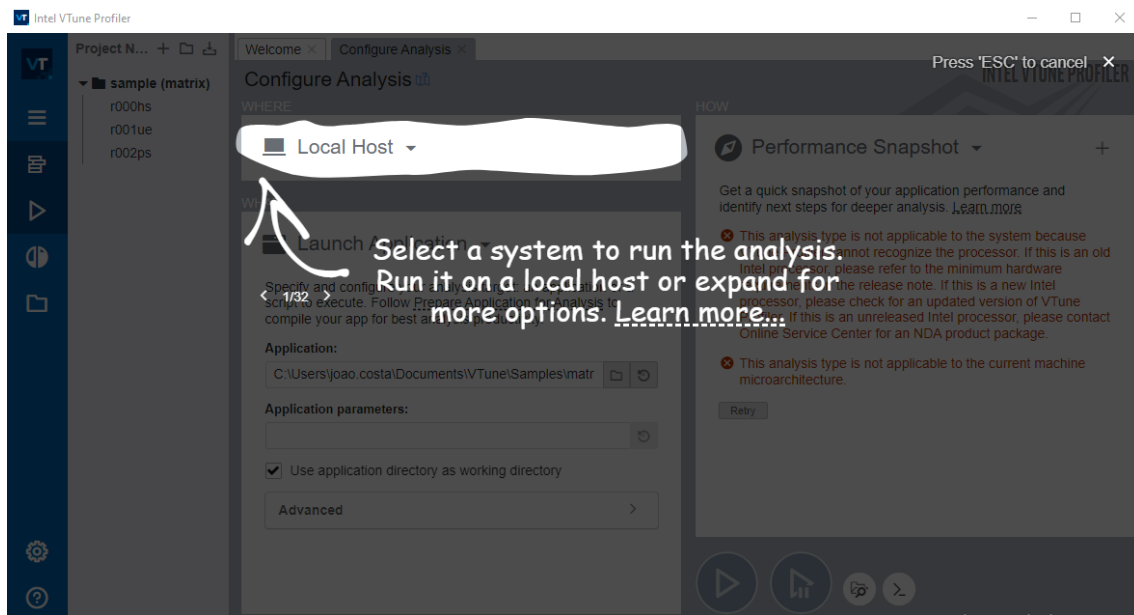
5. Instale o GCC v10 e os símbolos de debug para o GCC e o libgomp:

```
sudo apt-get install gcc-10 gcc-10-dbgsym libgomp1-dbgsym
```

6. Pronto! Agora, o Intel VTune Profiler deve apresentar as informações correctas sobre as directivas OpenMP.

B. Tutorial sobre o uso do Intel Vtune

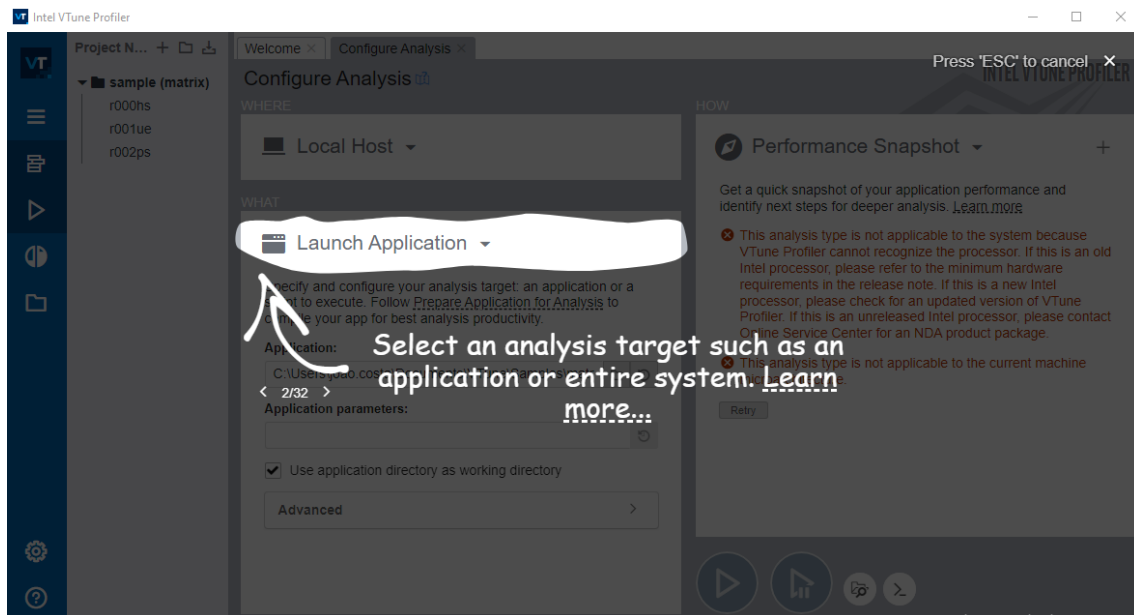




Use estas opções para decidir onde quer executar a análise:

Option	Description
Local Host	Run an analysis on the local host system.
Remote Linux (SSH)	Run an analysis on a remote regular or embedded Linux* system. VTune Profiler uses the SSH protocol to connect to your remote system.
Android Device (ADB)	Run an analysis on an Android device. VTune Profiler uses the Android Debug Bridge* (adb) to connect to your Android device.
Communication Agent (TCP/IP)	Profile an embedded system running a real-time operating system using the Analysis Communication Agent.
Arbitrary Host (not connected)	Create a command line configuration for a platform NOT accessible from the current host, which is called an <i>arbitrary target</i> .

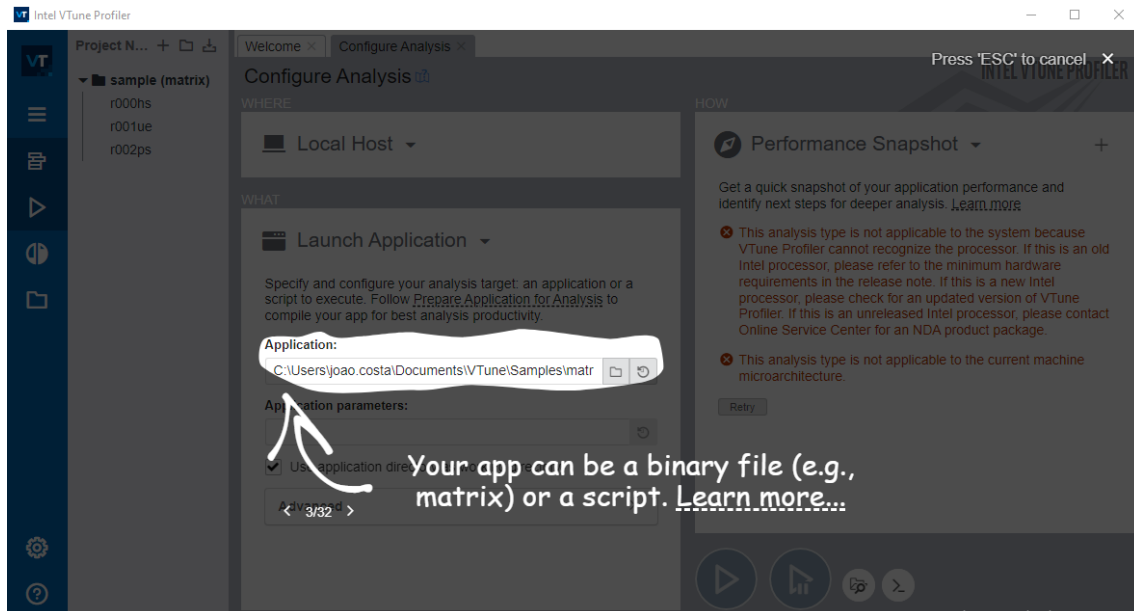
Leia mais sobre no link seguinte: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2025-1/analysis-system.html>

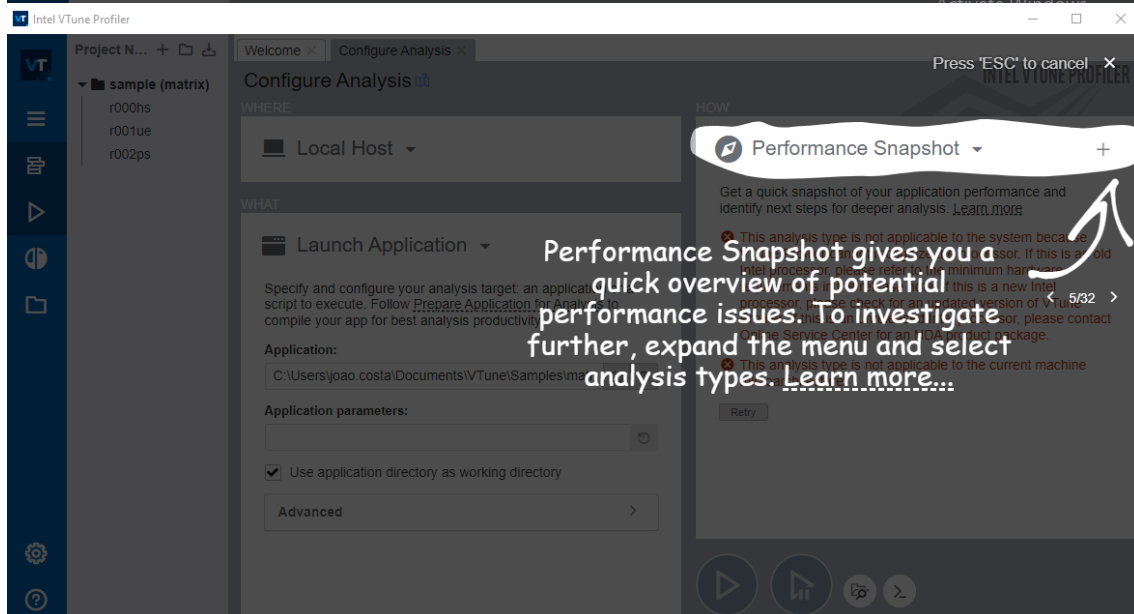
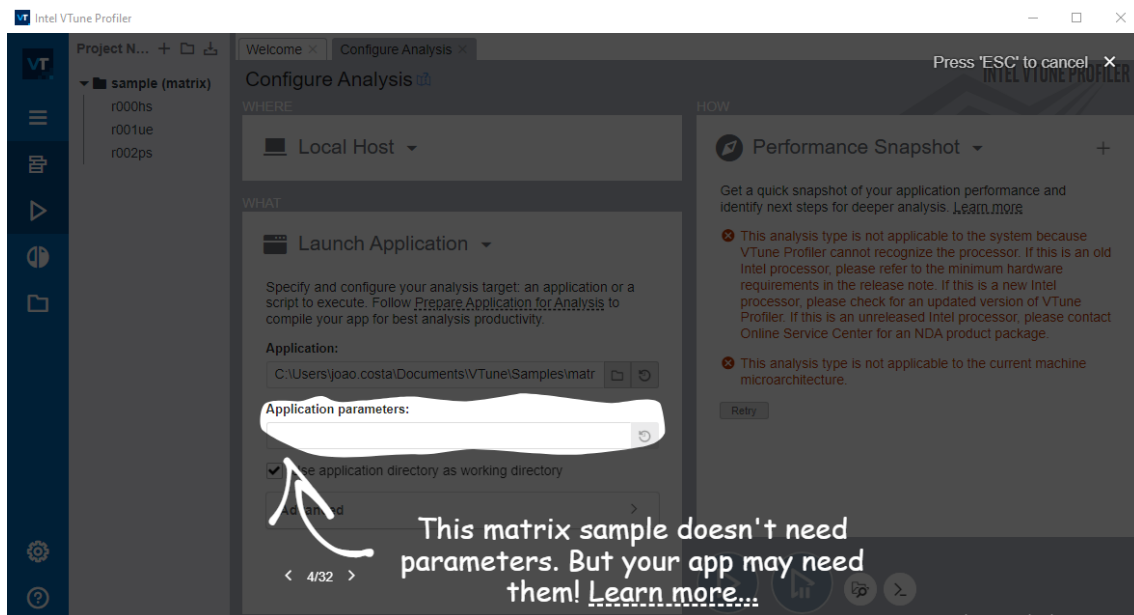


Um target é um ficheiro executável que analisa usando o Intel® VTune™ Profiler, que pode ser um ficheiro executável, um processo ou um sistema inteiro.

Por padrão, ao criar um novo projecto, o VTune Profiler abre uma configuração de análise com o destino de análise "Iniciar Aplicação" pré-seleccionado.

Leia mais sobre no link seguinte: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2025-1/analysis-target.html>





O VTune Profiler oferece diversos tipos de análise personalizados para examinar diversos tipos de aplicações e aspectos de desempenho. O Snapshot de Desempenho captura uma imagem desses aspectos e apresenta uma visão geral do funcionamento da sua aplicação.

Use o Snapshot de Desempenho quando quiser ver um resumo dos problemas que afectam sua aplicação. Esta análise também inclui recomendações para outros tipos de análise que pode executar em seguida para uma investigação mais aprofundada.

Leia mais sobre no link seguinte: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2025-1/performance-snapshot.html>

